

Password Cracking

Cybersecurity Internship – Phase 1, Task 8

Name: Parmar Rakesh

Date: 24, AUG, 2025

Internship Program: Cybersecurity Internship – UJAR TECH

How Does Password Cracking Work?

Before we dive into a practical example, you have to understand the basics of [password cracking\[25\]](#). The most popular techniques associated with password cracking are brute-force and dictionary attacks. It often comes after stealing critical data, such as databases that contain credentials. In the worst-case scenario, the credentials are in plain text, but most of the time you only get a hash.

Still, if the password is short and/or weak, John the Ripper can crack such data quickly. In general, the time required to crack passwords is directly linked to the strength of the password but, above all, its length.

That's why experts often say these kinds of attacks are about time and calculation power, but if you don't secure your passwords, they will be found at the speed of light. Or at least a good GPU.

What is a Hash?

A hash is a one-way cryptographic function that converts data (like a password) into a fixed-length string. It's irreversible by design—but with the right tools and wordlists, you can often guess the original input.

Common Hash Types

Here are typical hash types encountered in real-world scenarios:

- 1.MD5:** Fast but broken, common in older web apps
- 2.SHA1:** Slightly better than MD5, still insecure
- 3.bcrypt:** Slower and stronger (used in modern apps)
- 4.NTLM:** Used by Windows systems
- 5.SHA512 (crypt):** Linux password hashes in `/etc/shadow`

Identify the Hash Type

Sometimes hashes are labeled. If not, use tools like **hashid**.
Now, let's see how to install hashid in kali linux:

```
kejav@kejavi:~$ mkdir py
kejav@kejavi:~$ cd py
kejav@kejavi:~/py$ python3 -m venv zero
kejav@kejavi:~/py$ source zero/bin/activate
(zero)~kejav@kejavi:~/py$ pip install hashid
Collecting hashid
  Downloading hashid-3.1.4-py2.py3-none-any.whl.metadata (5.0 kB)
  Downloading hashid-3.1.4-py2.py3-none-any.whl (13 kB)
Installing collected packages: hashid
Successfully installed hashid-3.1.4
```

Here are typical hash types encountered in real-world scenarios:

- 1.MD5: Fast but broken, common in older web apps
- 2.SHA1: Slightly better than MD5, still insecure
- 3.crypt: Slower and stronger (used in modern apps)
- 4.NTLM: Used by Windows systems
- 5.SHA512 (crypt): Linux password hashes in /etc/shadow

Identify the Hash Type

Sometimes hashes are labeled. If not, use tools like: hashid (Python tool):

Activating Python Virtual Environment To Safe Testing

Install John the Ripper (JtR)

Install the community-enhanced version (Jumbo):

Linux (Debian/Ubuntu):

- **sudo apt install john**

For latest Jumbo version:

- **git clone https://github.com/openwall/john.git**
- **cd john/src**
- **./configure && make -s clean && make -sj4**

Use the run/ directory for binaries.

Cracking Linux Hashes

First we will make a hash just like shown in the following image using this command.

- **echo -n 'rakesh' | openssl passwd -1 -stdin > hash.txt**

```
(kejav@kejavi:~)$ echo -n 'rakesh' | openssl passwd -1 -stdin > hash.txt
(kejav@kejavi:~)$ cat hash.txt
$1$BrjYmzZ/$vWTksImGj2X9C0INjhYi/
```

Example.

```
6. user:$1$12345678$abcdefgEXAMPLEhas
7. Use John to crack the password
8. john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Now let's check how John cracked our hash. Here we can see that I created a hash of my name using openssl and stored the output into a hash.txt file. After that, I ran John using my rockyou.txt wordlist file. As you can see, within a short time John was able to crack the hash that I had generated earlier. This is how John works in practice.

- **john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt**

```
(kejav@kejav)~$ echo -n 'rakesh' | openssl passwd -1 -stdin > hash.txt
(kejav@kejav)~$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
rakesh (?)
1g 0:00:00:00 DONE (2025-08-25 02:37) 2.000g/s 52992p/s 52992c/s 52992C/s bearcat..dolphin5
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

```
(kejav@kejav)~$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
rakesh (?)
1g 0:00:00:00 DONE (2025-08-25 02:40) 2.380g/s 63085p/s 63085c/s 63085C/s bearcat..dolphin5
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

Task 7
(kejav@kejav)~$ john --show hash.txt
?:rakesh

Task 9
1 password hash cracked, 0 left
```